

A Hybrid Fish Swarm Optimisation Algorithm for Solving Examination Timetabling Problems

Hamza Turabieh¹ and Salwani Abdullah²

¹ Computer Science Department, Faculty of Science and Information Technology
Zarka University, Jordan
turabieh@zp.edu.jo

² Data Mining and Optimisation Research Group (DMO)
Center for Artificial Intelligence Technology,
Universiti Kebangsaan Malaysia, 43600 Bangi, Selangor, Malaysia
salwani@ftsm.ukm.my

Abstract. A hybrid fish swarm algorithm has been proposed to solve exam timetabling problems where the movement of the fish is simulated when searching for food inside water (refer as a search space). The search space is categorised into three categories which are **crowded**, **not crowded** and **empty areas**. The movement of fish (where the fish represents the solution) is determined based on a **Nelder-Mead simplex** search algorithm. The quality of the solution is enhanced using a **great deluge** algorithm or a **steepest descent** algorithm. The proposed hybrid approach is tested on a set of benchmark examination timetabling problems in comparison with a set of state-of-the-art methods from the literature. The experimental results show that the proposed hybrid approach is able to produce promising results for the test problem.

Keywords: Exam Timetabling, Fish Swarm Algorithm.

1 Introduction

Timetabling problems present a challenging problem area for researchers across both operational research and artificial intelligence. This kind of problems can be classified as scheduling problems. This concept can be defined based on Fox and Sadeh-Konieczpol [15] as follows:

“Scheduling selects among alternative plans, and assigns resources and times to each activity so that they obey the temporal restrictions of activities and the capacity limitations of a set of shared resources.”

The examination timetabling problem is a popular problem in the academic world for schools or higher educational institutes which are concerned with allocating exams into a limited number of time slots (periods) subject to a set of constraints so that no students should sit for two or more exams at the same time and the scheduled exams must not exceed the room capacity.

Up to date, many approaches have been introduced to solve examination timetabling problems. Carter [9] categorised these approaches into four types: sequential methods, cluster methods, constraint-based methods and generalised search. Petrovic and Burke [26] added the following categories: hybrid evolutionary algorithms, meta-heuristics, multi-criteria approaches, case based reasoning techniques, hyper-heuristics and adaptive approaches.

Graph colouring heuristics methods seem to be the earliest algorithms applied in this problem, followed by stochastic search methods such as simulated annealing, tabu search, genetic algorithm etc., that are considered as meta-heuristic approaches. In general, these approaches can be classified into two categories i.e. single-based approach and population-based approach. Single-based approach such as simulated annealing [2], large neighbourhood search [1] and tabu search [17] in general works on a single solution and try to find a better solution in a solution space. On the other hand, population-based approaches start with many solutions and then try to obtain optimal solution(s) in the whole search space. The most common population-based approaches are evolutionary algorithms [11], ant colony algorithms [12], artificial immune systems [22] etc. In this work, we deal with a hybrid population-based approach that combine the good properties of a single-based approach (a great deluge algorithm in this case) and a population-based approach (i.e. fish swarm algorithm) to solve uncapacitated examination timetabling problem. We try to create a balance between the exploration (generated by the fish swarm algorithm) and exploitation (generated by the great deluge algorithm). These algorithms are discussed in details in Section 3 and Section 4.2, respectively. Interested readers can refer to [20], [23], [29] [30], and [31] for a comprehensive survey on timetabling.

Will need to search these up

The rest of the paper is organised as follows. The next section describes the examination timetabling problem in details. Section 3 presents a fish swarm algorithm followed by the hybrid approach in Section 4. Experimental results of comparing the proposed hybrid approach with other algorithms from the literature are reported and discussed in Section 5. Finally, Section 6 concludes this paper with brief concluding comments and discussion on the future work.

2 Uncapacitated Examination Timetabling Problem

In this work, the formulations of examination timetabling problem have been adapted from the description presented in Burke et al. [6]. The problem consists of:

- E_i is a collection of N examinations ($i=1, \dots, N$).
- T is the number of timeslots.
- $C=(c_{ij})_{N \times N}$ is the conflict matrix where each record, denoted by c_{ij} ($i, j \in \{1, \dots, N\}$), represents the number of students taking exams i and j .
- M is the number of students.
- t_k ($1 \leq t_k \leq T$) specifies the assigned timeslots for exam k ($k \in \{1, \dots, N\}$).

The specifications of the uncapacitated examination timetabling problems are shown in Table 1 as taken from [29].

Table 1. Specifications of the uncapacitated examination timetabling problem

Datasets	Number of timeslots	Number of examinations	Number of students	Conflict matrix density
car91	32	543	18419	0.14
car92	35	682	16925	0.13
ear83I	24	190	1125	0.29
hec92I	18	81	2823	0.42
kfu93	20	461	5349	0.06
lse91	18	381	2726	0.06
sta83I	13	139	611	0.14
tre92	23	261	4360	0.18
uta92I	35	622	21267	0.13
ute92	10	184	2750	0.08
yor83I	21	181	941	0.27

Only one hard constraint is considered for uncapacitated exam timetabling which is no students should be required to sit two examinations simultaneously.

In this problem, we formulate an **objective function** which tries to spread out students' exams throughout the exam period (Expression (1)) that is treated as a soft constraint.

Our objective function will be: $J(N, G, C)$
 N is the set of students,
 G is the set of gap length of the timetable for each particular student,
 C is the average credit of the course
 (to determine how hard the course was over time),
 or simply the number of credit hours of that course

$$\min \frac{\sum_{i=1}^{N-1} F(i)}{M} \quad (1)$$

Where:

Will need to dive deeper into the math behind this,
 but will first go through what proximity functions are.

$$F(i) = \sum_{j=i+1}^N c_{ij} \cdot \text{proximity}(t_i, t_j) \quad (2)$$

$$\text{proximity}(t_i, t_j) = \begin{cases} 2^5 / 2^{|t_i - t_j|} & \text{if } 1 \leq |t_i - t_j| \leq 5 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

Subject to:

If the student is conflicting (taking both exams)
 And they are in the same timeslot
 The sum of that should equal zero, in other words, all students should not have any clash, if they do, the value of the summation will be greater than 0, thus breaking the hard constraint and the timetable will not be valid

$$\sum_{i=1}^{N-1} \sum_{j=i+1}^N c_{ij} \cdot \lambda(t_i, t_j) = 0 \quad \text{where} \quad \lambda(t_i, t_j) = \begin{cases} 1 & \text{if } t_i = t_j \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

Loop over all exams
 Loop over all exams other than the current one

Equation (2) presents the cost for an exam i which is given by the proximity value multiplied by the number of students in conflict. Equation (3) represents a proximity value between two exams [8]. Equation (4) represents a clash-free requirement so that no student is asked to sit two exams at the same time. The clash-free requirement is considered to be a hard constraint.

3 The Fish Swarm Optimisation Algorithm

Studying the behavior of fish through searching for food attracts a lot of researchers due to its ability to find foods in a large searching area. In this section, we present the idea of fish swarming algorithm, that simulates the behavior of a fish inside water (search

space) while they are searching for food. Fish swarm algorithm has been successfully applied on many optimisation problem such as in [14], [16], [18], [19], [21], [32], and [33]. In this work, the fish swarm algorithm works on selected solutions rather than all solutions, with an aim to reduce the computational time. We will use the words “fish” and “solution” interchangeably throughout the paper. The basic point of a fish swarm algorithm is the visual scope (area). We categorised three possible cases for the visual scope for Sol_i (where Sol represents a solution) as in Figure 1:

- Empty visual: no solution is closed to Sol_i .
- Crowded visual: many solutions are closed to Sol_i .
- Not crowded visual: a few solutions are closed to Sol_i .

A visual scope for a selected solution Sol_i can be represented as the scope of closed solutions. The determination of the number of solutions that are closed to Sol_i is based on the number of solutions inside the visual scope. The visual is used to determine the closeness of two solutions which is based on the distance (solution quality) between two solutions (i.e. $f(x') - f(x)$). Note that in this work, if the distance is less or equal to 10, then the solution x' is closed to x . The category of the visual scope is determined based on the following mechanism:

$$\text{Loss} = f(\text{predicted}) - f(\text{target})$$

$$\frac{\text{Number of closed solutions}}{\text{size of population}} \leq \theta$$

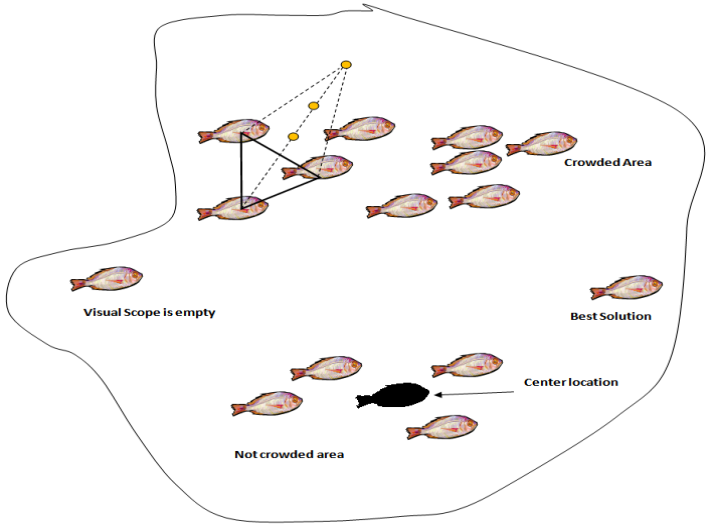


Fig. 1. A representation of solutions in the search space

where θ is set to 0.5. Note that all the constant values used in this algorithm are obtained through some preliminary experiments. If the number of the closed solution is more than the size of the visual scope, then the visual scope is considered as a crowded area. The size of visual scope is calculated as:

$$\frac{\text{Size of Population}}{2} + 1$$

i.e. in other words, the visual scope is considered as a crowded area if the number of the closed solution inside this visual scope is at least greater than half of the population size.

The simulation behavior of the fish swarm algorithm can be described in three steps i.e. swarming, chasing and searching behaviors which are based on the category of the visual scope as shown in Figure 2. If the visual scope is a crowded area then a searching behavior is employed; if the visual scope is a not crowded area then a swarming or a chasing behavior is applied; if the visual scope is empty, then a chasing behavior is applied. The details of the simulation behavior of the fish swarm algorithm that is employed in this work are discussed as below:

Input

m: size of population

Set Iteration =1

Initialization Phase

$(X_1, X_2, \dots, X_m) \leftarrow \text{Construction}();$

Improvement Phase

While termination condition is not satisfied do

 Select a solution using Roulette Wheel Selection (RWS) Sol_i

 Set best solution as Sol_{best} ;

 Compute the visual for Sol_i

 if visual scope is an empty area then

$Sol_i' = \text{Steepest descent}(Sol_i)$

 else

 if visual scope is a crowded area then

 Apply a multi-decay rate great deluge to obtain Sol_i'
 as in Fig 5.

 else

 Central Location = compute the central point of visual

 if Central Location is better than Sol_i then

 Estimated quality = Central Location;

 Apply a standard great deluge (swarming
 behavior) as in [13] to obtain Sol_{iA} ;

 else

 Estimated quality = Best Solution, Sol_{best} ;

 Apply a standard great deluge (chasing behavior)
 as in [13] to obtain Sol_{iB} ;

 end if

 if $Sol_{best} < Sol_i$ then

 Estimated quality = Best Solution, Sol_{best} ;

 Apply a standard great deluge (chasing behavior)
 as in [13] to obtain Sol_{iB} ;

 else

$Sol_{iB} = \text{Steepest descent}(Sol_i)$

 end if

$Sol_i' = \min\{Sol_{iA}, Sol_{iB}\}$

 end if

$Sol_i'' = \min\{Sol_i, Sol_i'\}$ // Update solution

 Iteration++

 end while

Fig. 2. The pseudo code for the fish swarm algorithm

A. *Swarming or Chasing behavior*

A fish (solution) swarms towards a central point (which is an average of the *visual scope*) of the search area. This behavior is applied if and only if the central point (in terms of the quality of the solution) is better than the selected solution Sol_i . Otherwise, a fish will chase a best solution so far. These behaviors are represented by a great deluge algorithm (see Section 4.2) where a central point or a best solution is treated as an estimated quality, respectively. However, if the *visual scope* is empty, a chasing behavior that employed a steepest descent algorithm is employed.

B. *Searching behaviour*

In this behavior, a Nelder-Mead simplex algorithm [21] (as in Section 4.1) is used to determine the movement directions of the fish. There are three directions called a Contraction-External (CE), a Reflection (R) and an Expansion (E). These movement directions are later to be used to intelligently control the multi decay rate in the great deluge algorithm. Note that the details on the multi decay rate great deluge algorithm are discussed in Section 4.2.

4 The Hybrid Approach

This means that it is converging to a local minimum and not a near-optimal global minimum, this can be optimized using momentum.

The main issue of hybridising a fish swarm algorithm and the great deluge (or steepest descent) algorithm is to combine the advantages these algorithms. Before the great deluge is employed, a Nelder-Mead simplex method is used to calculate the expected location (estimated quality value) of the selected solution in the population. Nelder-Mead simplex method is a very efficient local search method procedure but its convergence is extremely sensitive to the selected starting point, thus a great deluge algorithm is hybridised in order to control the convergence by using a multi decay rate that control the level of accepting a worse solution during the search process. The following two subsections will describe Nelder-Mead simplex algorithm and a multi decay rate great deluge algorithm, respectively.

4.1 Nelder-Mead Simplex Algorithm

We are working on constraint, does that means that this is not applicable

Nelder-Mead simplex algorithm has been proposed by Nelder and Mead [25] which is a local search method designed for unconstrained optimisation without using gradient information. This algorithm tries to find an approximation of a local optimum of a problem with N variables. The method is based on the theory of simplex, which is a special polytope of $N + 1$ vertices in N dimensions. The operations of this method rescale the simplex based on the local behavior of the function by using four basic procedures: reflection, expansion, contraction external and shrinkage. In this work only three values have been estimated based on Nelder-Mead simplex algorithm which they are as follows:

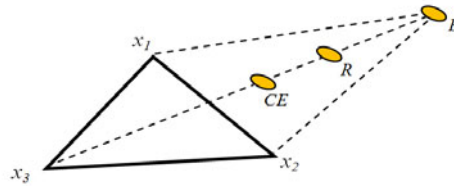


Fig. 3. Nelder-Mead simplex search algorithm

- $\text{Contraction_External}(CE) = \left[\frac{x_1 + x_2}{2} \right] - \left[\frac{\left[\frac{x_1 + x_2}{2} \right] + x_3}{2} \right]$
 - $\text{Reflection}(R) = \text{Contraction_External} - \left[\frac{\left[\frac{x_1 + x_2}{2} \right] + x_3}{2} \right]$
 - $\text{Expansion}(E) = \text{Reflection} - \left[\frac{\left[\frac{x_1 + x_2}{2} \right] + x_3}{2} \right]$
- Didn't really understand what these represent, will read the paper referenced

The three expected locations (estimated values) will be used within the multi decay rate great deluge algorithm in order to control the great deluge decay rate in accepting a worse solution. Figure 3 represents the three possible new locations for the x_3 in the search space (i.e. at CE , R or E).

4.2 Multi Decay Rate Great Deluge Algorithm

In this work, we applied a great deluge algorithm [13] that can intelligently control the decay rate based on the estimated value calculated using Nelder-Mead simplex algorithm. The great deluge algorithm always accepts a better solution, and a worse solution is accepted in order to escape from local minimum. The acceptance of the worse solution is controlled by the decay rate that later will affect the performance of searching behavior. Figure 4 illustrates the changing of the decay rate at different estimated values.

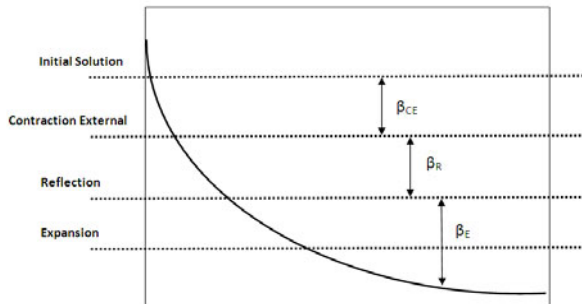


Fig. 4. Changing decay rate at different estimated values (Contraction-External, Reflection and Expansion)

Figure 5 represents the pseudo code of the proposed multi decay rate great deluge algorithm.

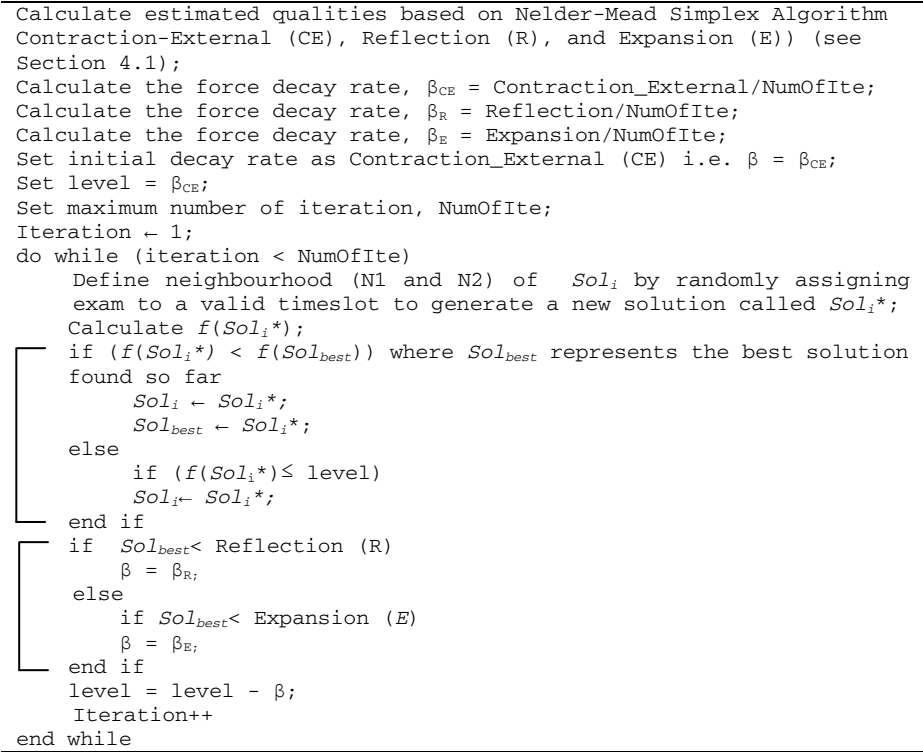


Fig. 5. The pseudo code for the multi decay rate great deluge algorithm

Two different neighbourhood structures and their explanation are outlined as follows:
N1: Select two exams at random and swap timeslots.
N2: Choose a single exam at random and move to a new random feasible timeslots.

5 Simulation Results We don't really need much information from this now,
Might come back later after finishing the algorithm,
But I will go through it quickly

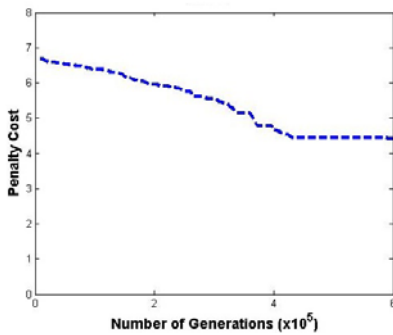
In order to test our proposed algorithm, we employed it over uncapacitated examination timetabling datasets that was introduced by Carter et al. [8]. The details of datasets can also be found in [29]. The proposed algorithm was programmed using Matlab and simulations were performed on the **Intel Pentium 4 2.33 GHz computer**. The parameter settings used in this work are shown in Table 2. We still have so much more computing power

Table 2. Parameters setting

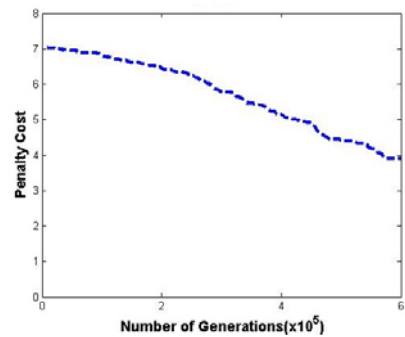
Parameter	Iteration	Population size	GD-iteration	Steepest descent iteration	Visual
Value	1000000	50	2000	2000	10

Table 3. Results

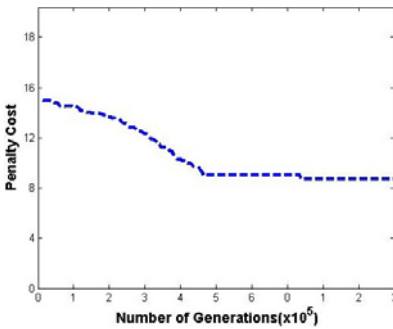
Instance	Best	Avg.	Median	Q1	Q3
car91	4.81	4.93	4.90	4.84	4.92
car92	4.11	4.33	4.19	4.17	4.53
ear83I	36.10	36.54	36.47	36.33	35.58
hec92I	10.95	11.34	11.01	11.00	11.36
kfu93	13.21	13.43	13.60	13.33	13.75
lse91	10.20	10.54	10.65	10.51	10.93
sta83I	159.74	159.93	159.82	159.81	159.96
tre92	8.00	8.45	8.36	8.35	8.51
uta92I	3.32	3.62	3.53	3.33	3.70
ute92	26.17	26.77	26.65	26.55	26.81
yor83I	36.23	36.55	36.42	36.33	36.71



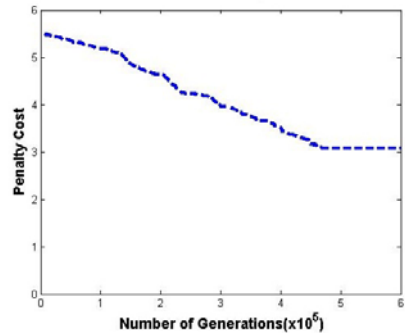
a)



b)



c)



d)

Fig. 6. Convergence using fish swarm optimization algorithm (a) car91 dataset convergence; (b) car92 dataset convergence; (c) tre92 dataset convergence; (d) ute92 dataset convergence.

The experiment carried out in this section attempts to space out students' examination throughout the examination period. Termination condition is the number of generations (i.e. 1,000,000 generations). The best, average, median together with the upper and lower quartiles results are reported in Table 3 which is out of 11 runs.

From Table 3, we can see that our approach is able to obtain good enough results since the difference between best and average for all datasets is in between (0.08 to 0.35) except for ear83I. We believe the algorithm is robust since the difference between best and average is small and also the difference between the upper and lower quartiles is in between (0.03 to 0.44), again except for ear83I. This shows that the solutions obtained are close to each other at the solution search space.

Figures 6 (a), (b), (c) and (d) show the behavior of our algorithm when exploring the search space on car91, car92, tre92 and ute92 datasets, respectively. The *x*-axis represents the number of generations, while the *y*-axis represents the overall penalty cost. These graphs demonstrate how the hybrid algorithm explores the search space. Note that the timetable quality is measured by taking the average penalty per student. The curves show that the algorithm begins with initial solutions and rapidly improves the quality of the timetable. For most of the cases (except for car92), the graphs show no improvement on the quality of solutions at about 200000 iterations (for *tre92*) and 400000 iterations (for *car91* and *ute92*). For the *car92* dataset, the algorithm is still able to reduce the penalty cost even after 1000000 iterations. This shows that given extra processing time, the quality of solution on certain datasets might be able to be enhanced further.

Table 4. Comparison of our results with other approaches in the literature

Instance	Our approach	Merlot et al. [24]	Casey and Thompson [10]	Côté et al. [11]	Yang and Petrovic [34]	Turabieh and Abdullah [3]
<i>car91</i>	4.81	5.1	5.4	5.2	4.50	4.80
<i>car92</i>	4.11	4.3	4.2	4.2	3.93	4.10
<i>ear83I</i>	36.10	35.1	34.2	34.2	33.7	34.92
<i>hec92I</i>	10.95	10.6	10.2	10.2	10.83	10.73
<i>kfu93</i>	13.21	13.5	14.2	14.2	13.82	13.00
<i>lse91</i>	10.20	10.5	14.2	11.2	10.35	10.01
<i>sta83I</i>	159.74	157.3	134.9	157.2	158.35	158.26
<i>tre92</i>	8.00	8.4	8.2	8.2	7.92	7.88
<i>uta92I</i>	3.32	3.5	-	3.2	3.14	3.20
<i>ute92</i>	26.17	25.1	25.2	25.2	25.39	26.11
<i>yor83I</i>	36.23	37.4	37.2	36.2	36.35	36.22

Instance	Caramia et al. [7]	Abdullah and Burke [1]	Qu and Burke [28]	Qu et al. [27]	Burke et al. [5]	Burke and Bykov [4]
<i>car91</i>	6.6	4.1	5.16	5.11	4.6	4.42
<i>car92</i>	6.0	4.8	4.16	4.32	3.9	3.74
<i>ear83I</i>	29.3	36.0	35.86	35.56	32.8	32.76
<i>hec92I</i>	9.2	10.8	11.94	11.62	10.0	10.15
<i>kfu93</i>	13.8	15.2	14.79	15.18	13.0	12.96
<i>lse91</i>	9.6	11.9	11.15	11.32	10.0	9.83
<i>sta83I</i>	158.2	159.0	159	158.88	156.84	157.03
<i>tre92</i>	9.4	8.5	8.6	8.52	7.9	7.75
<i>uta92I</i>	3.5	3.6	3.59	3.21	3.2	3.06
<i>ute92</i>	24.4	26.0	28.3	28	24.8	24.82
<i>yor83I</i>	36.2	36.2	41.81	40.71	34.9	34.84

Table 4 shows the comparison of our final results in terms of penalty cost compared to other recent published results in the literature. The best results are presented in bold. Our algorithm is capable to find a good enough feasible timetables for all eleven cases. Even though, our algorithm is not able to obtain new best results, but from Table 4 we can see that the algorithm works reasonably well across all datasets and we did not perform the worse in any of the datasets, even the complexity of the datasets are different (based on the conflict density value). We believe this is because the capability of the proposed hybrid approach that can make a balance between exploration and exploitation during the search process that help to minimise the objective function values and give competitive results for the uncapacitated examination timetabling problem compared to other algorithm in the literature.

6 Conclusion and Future Work

In this paper, a simulation of fish swarm algorithm has been applied on uncapacitated exam timetabling problems. The search space has been categorised into three categories, which are crowded, not crowded and empty areas. The movement of each fish (solution) is based on its location in the search space. Intelligent decay rate have been proposed for great deluge algorithm based on the hybridisation with the Nelder–Mead simplex algorithm. The obtained results are good enough and we strongly believe that categorising the search space enhances the exploration process while intelligent controlling the decay rate enhances the exploitation process. **Our future work will aim to test this algorithm on International Timetabling Competition dataset (ITC2007) for both exam and course timetabling problems.** Did they?

References

- [1] Abdullah, S., Burke, E.K.: A Multi-start large neighbourhood search approach with local search methods for examination timetabling. In: International Conference on Automated Planning and Scheduling (ICAPS 2006), Cumbria, UK, pp. 334–337 (2006)
- [2] Abdullah, S., Shaker, K., McCollum, B., McMullan, P.: Dual sequence simulated annealing with round-robin approach for university course timetabling. In: Cowling, P., Merz, P. (eds.) EvoCOP 2010. LNCS, vol. 6022, pp. 1–10. Springer, Heidelberg (2010)
- [3] Turabieh, H., Abdullah, S.: An integrated hybrid approach to the examination timetabling problem. OMEGA (2011), doi:10.1016/j.omega.2010.12.005
- [4] Burke, E.K., Bykov, Y.: Solving exam timetabling problems with the flex-deluge algorithm. In: Burke, E.K., Rudová, H. (eds.) PATAT 2007. LNCS, vol. 3867, pp. 370–372. Springer, Heidelberg (2007) ISBN: 80-210-3726-1
- [5] Burke, E.K., Eckersley, A.J., McCollum, B., Petrovic, S., Qu, R.: Hybrid variable neighbourhood approaches to university exam timetabling. European Journal of Operational Research 206, 46–53 (2010)
- [6] Burke, E.K., Kingston, J., de Werra, D.: Applications to timetabling. In: Gross, J., Yellen, J. (eds.) Handbook of Graph Theory, pp. 445–474. Chapman Hall/CRC Press (2004)

- [7] Caramia, M., Dell'Olmo, P., Italiano, G.F.: New algorithms for examination timetabling. In: Näher, S., Wagner, D. (eds.) WAE 2000. LNCS, vol. 1982, pp. 230–241. Springer, Heidelberg (2001)
- [8] Carter, M.W., Laporte, G., Lee, S.: Examination timetabling: Algorithmic strategies and applications. *Journal of the Operational Research Society* 47(3), 373–383 (1996)
- [9] Carter, M.W.: A survey of practical applications of examination timetabling algorithms. *Operations Research* 34(2), 193–202 (1986)
- [10] Casey, S., Thompson, J.: GRASping the examination scheduling problem. In: Burke, E.K., De Causmaecker, P. (eds.) PATAT 2002. LNCS, vol. 2740, pp. 232–244. Springer, Heidelberg (2003)
- [11] Côté, P., Wong, T., Sabourin, R.: A hybrid multi-objective evolutionary algorithm for the uncapacitated exam proximity problem. In: Burke, E.K., Trick, M.A. (eds.) PATAT 2004. LNCS, vol. 3616, pp. 294–312. Springer, Heidelberg (2005)
- [12] Dowsland, K., Thompson, J.: Ant colony optimization for the examination scheduling problem. *Journal of the Operational Research Society* 56(4), 426–438 (2005)
- [13] Dueck, G.: New Optimization Heuristics. The great deluge algorithm and the record-to-record travel. *Journal of Computational Physics* 104, 86–92 (1993)
- [14] Fernandes, E.M.G.P., Martins, T.F.M.C., Rocha, A.M.A.C.: Fish Swarm Intelligent Algorithm for Bound Constrained Global Optimization. In: *Proceedings of the International Conference on Computational and Mathematical Methods in Science and Engineering, CMMSE 2009*, June 30, July 1–3 (2009)
- [15] Fox, M.S., Sadeh-Konieczpol, N.: Why is scheduling so difficult? A csp perspective. In: *Proceedings of the European Conference on Artificial Intelligence*, pp. 754–767 (1990)
- [16] Gao, S., Yang, J.Y.: *Swarm intelligence algorithms and applications*. China Waterpower Press, Beijing (2006)
- [17] Gaspero, L.D., Schaerf, A.: Tabu search techniques for examination timetabling. In: Burke, E., Erben, W. (eds.) PATAT 2000. LNCS, vol. 2079, pp. 104–117. Springer, Heidelberg (2001)
- [18] Jiang, M., Mastorakis, N., Yuan, D., Lagunas, M.A.: Image segmentation with improved artificial fish swarm algorithm. In: Mastorakis, N., Mladenov, V., Kontargyri, V.T. (eds.) *Proceedings of the European Computing Conference. Lecture Notes in Electrical Engineering*, vol. 28, pp. 133–138. Springer, Heidelberg (2009) ISBN: 978-0-387-84818-1
- [19] Jiang, M., Wang, Y., Pfletschinger, S., Lagunas, M.A., Yuan, D.: Optimal multiuser detection with artificial fish swarm algorithm. In: Huang, D.-S., Heutte, L., Loog, M. (eds.) ICIC 2007. CCIS, vol. 2, pp. 1084–1093. Springer, Heidelberg (2007)
- [20] Lewis, R.: A survey of metaheuristic-based techniques for university timetabling problems. *OR Spectrum* 30(1), 167–190 (2008)
- [21] Li, X.L., Shao, Z.J., Qian, J.X.: An optimizing method based on autonomous animate: fish swarm algorithm. *System Engineering Theory and Practice* 11, 32–38 (2002)
- [22] Malim, M.R., Khader, A.T., Mustafa, A.: Artificial immune algorithms for university. In: Burke, E.K., Rudová, H. (eds.) PATAT 2007. LNCS, vol. 3867, pp. 234–245. Springer, Heidelberg (2007)
- [23] McCollum, B.: A perspective on bridging the gap between theory and practice in university timetabling. In: Burke, E.K., Rudová, H. (eds.) PATAT 2007. LNCS, vol. 3867, pp. 3–23. Springer, Heidelberg (2007)
- [24] Merlot, L.T.G., Boland, N., Hughes, B.D., Stuckey, P.J.: A Hybrid Algorithm for the Examination Timetabling Problem. In: Burke, E.K., De Causmaecker, P. (eds.) PATAT 2002. LNCS, vol. 2740, pp. 207–231. Springer, Heidelberg (2003)

- [25] Nelder, J.A., Mead, R.: A simplex method for function minimization. *Computer Journal* 7, 308–313 (1965)
- [26] Petrovic, S., Burke, E.K.: Chapter 45: University timetabling. In: Leung, J. (ed.) *Handbook of Scheduling: Algorithms Models and Performance Analysis*. CRC Press, Boca Raton (2004)
- [27] Qu, R., Burke, E.K., McCollum, B.: Adaptive automated construction of hybrid heuristics for exam timetabling and graph colouring problems. *European Journal of Operational Research (EJOR)* 198(2), 392–404 (2009)
- [28] Qu, R., Burke, E.K.: Hybridisations within a graph based hyper-heuristic framework for university timetabling problems. *Journal of Operational Research Society (JORS)* 60, 1273–1285 (2009)
- [29] Qu, R., Burke, E.K., McCollum, B., Merlot, L.T.G., Lee, S.Y.: A survey of search methodologies and automated system development for examination timetabling. *Journal of scheduling*, 55–89 (2009)
- [30] Sadeh, N., Kaujunn, M.: Micro-opportunistic scheduling: The micro-boss factory scheduler. In: *Intelligent Scheduling*, pp. 99–135. Morgan Kaufmann, San Francisco (1994)
- [31] Schaerf, A.: A survey of automated timetabling. *Artificial Intelligence Review* 13(2), 87–127 (1999)
- [32] Wang, C.-R., Zhou, C.-L., Ma, J.-W.: An improved artificial fish swarm algorithm and its application in feed-forward neural networks. In: *Proceedings of the Fourth International Conference on Machine Learning and Cybernetics*, pp. 2890–2894 (2005)
- [33] Wang, X., Gao, N., Cai, S., Huang, M.: An Artificial Fish Swarm Algorithm Based and ABC Supported QoS Unicast Routing Scheme in NGI. In: Min, G., Di Martino, B., Yang, L.T., Guo, M., Rünger, G. (eds.) *ISPA Workshops 2006*. LNCS, vol. 4331, pp. 205–214. Springer, Heidelberg (2006)
- [34] Yang, Y., Petrovic, S.: A Novel Similarity Measure for Heuristic Selection in Examination Timetabling. In: Burke, E.K., Trick, M.A. (eds.) *PATAT 2004*. LNCS, vol. 3616, pp. 247–269. Springer, Heidelberg (2005)